MiniLM Project 1

A Logistic-Regression Next-Token Language Model

Part 1 of 3: Logistic Regression → RNN → Transformer

Course	Applied Natural Language Processing
Language	Python 3
Recommended duration	5 days

This is a small educational language-modeling pipeline. Students may use appropriate Python libraries and are not expected to reimplement established algorithms from scratch.

1. Three-Project Sequence

Across three connected projects, you will build and compare three next-token prediction models that use the same corpus, vocabulary conventions, prediction task, evaluation ideas, and generation workflow.

Project	Model	Primary idea	Connection to the sequence
1	Multinomial logistic regression	Sparse context features and a transparent baseline	Creates the shared data-processing and evaluation pipeline
2	Recurrent neural network	Learned embeddings and sequential hidden state	Replaces the baseline model while preserving the next-token task
3	Transformer	Self-attention, positional information, and causal prediction	Compares all three models in a miniature language-model application

2. Project Goal and Library Policy

Build a next-token prediction model using multinomial logistic regression. Given the previous three tokens, the model predicts the most likely next token and can be called repeatedly to generate short text sequences.

Library policy: You may use standard Python libraries and the libraries listed in this handout. You are not expected to implement tokenizers, sparse-matrix data structures, optimization algorithms, logistic regression, or evaluation metrics from scratch. You are responsible for preparing data correctly, selecting and calling appropriate library tools, connecting the pipeline, testing the results, and explaining model behavior.

- Use the provided train, validation, and test splits.
- Build the vocabulary from training text only.
- Create fixed-length contexts without crossing CSV-row boundaries.
- Evaluate accuracy, top-3 accuracy, and log loss.
- Generate text with greedy and temperature-based decoding.
- Identify in the reflection which pipeline components can be reused conceptually in Projects 2 and 3.

3. Data and Modeling Rules

The provided mini_language_corpus.csv contains 120 short texts: 90 training, 15 validation, and 15 test rows.

```
text_id,split,text
t001,train,the student explains the short document carefully.
t091,validation,the student explains the short document carefully.
t106,test,the programmer generates the text sequence before evaluation.
```

- Do not randomly resplit the corpus.
- Lowercase text and tokenize words plus sentence-ending punctuation. A Python library or deterministic regular expression may be used.
- Add <bos> at the beginning and <eos> at the end of each text.
- Reserve IDs 0, 1, 2, and 3 for <pad>, <unk>, <bos>, and <eos>.
- Use <unk> for tokens not retained in the training vocabulary.
- Use a context window of 3 tokens.
- Use random_state=42 whenever an applicable library function accepts a seed.

4. Google Colab Setup

- Create a Python 3 notebook in Google Colab.
- Upload mini_language_corpus.csv using the Files panel.
- Run the installation command before importing the libraries.

```
!pip install -q pandas numpy scipy scikit-learn
import re
import pandas as pd
import numpy as np
from scipy.sparse import csr_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, top_k_accuracy_score, log_loss
```

- Run the notebook from top to bottom before submission.
- Export the final implementation as mini_lm_project1.py.
- Colab uploads are temporary; upload the CSV again after a runtime restart.

5. Required Functions

The following functions define expected inputs, outputs, and behavior. They do not require from-scratch implementations of existing library algorithms. Each complete function explanation begins on a new page so the explanation is never split across pages.

load_corpus(csv_file)

Required behavior and suggested library tools are listed below. The entire function specification is kept together on this page.

Parameters	Path to mini_language_corpus.csv
Returns	pandas.DataFrame
Requirements	Return exactly the columns text_id, split, and text. Strip surrounding whitespace, preserve row order, and reject blank text or a split outside train, validation, and test.
Suggested tools	pandas.read_csv and DataFrame operations.

tokenize(text)

Required behavior and suggested library tools are listed below. The entire function specification is kept together on this page.

Parameters	A string
Returns	list[str]
Requirements	Lowercase the text and return word tokens plus sentence-ending punctuation. Do not return whitespace or empty tokens. The result must be deterministic.
Suggested tools	re.findall, a scikit-learn text utility, or another deterministic Python tokenization approach. Do not use a pretrained language model or external web service.

build_vocabulary(train_texts, min_frequency=1)

Required behavior and suggested library tools are listed below. The entire function specification is kept together on this page.

Parameters	Iterable of training strings and a minimum frequency
Returns	tuple: token_to_id, id_to_token
Requirements	Include <pad>, <unk>, <bos>, and <eos> first with IDs 0, 1, 2, and 3. Add retained training tokens in alphabetical order. Build from training text only.
Suggested tools	collections.Counter, pandas, NumPy, scikit-learn utilities, or standard Python collections.

encode_text(text, token_to_id)

Required behavior and suggested library tools are listed below. The entire function specification is kept together on this page.

Parameters	Text and vocabulary mapping
Returns	list[int]
Requirements	Return [<bos>, token IDs..., <eos>]. Replace any unknown token with the <unk> ID.
Suggested tools	The tokenize function and standard Python sequence operations.

make_examples(texts, token_to_id, context_size=3)

Required behavior and suggested library tools are listed below. The entire function specification is kept together on this page.

Parameters	Iterable of texts, vocabulary mapping, and context size
Returns	tuple: contexts, targets
Requirements	Create one target for every token after <bos>. Left-pad short contexts with <pad>. Return contexts as a 2D integer NumPy array and targets as a 1D integer NumPy array. Never cross text boundaries.
Suggested tools	NumPy, pandas, standard Python sequence operations, and the required helper functions.

vectorize_contexts(contexts, vocabulary_size)

Required behavior and suggested library tools are listed below. The entire function specification is kept together on this page.

Parameters	2D context-ID array and vocabulary size
Returns	scipy.sparse.csr_matrix
Requirements	Create bag-of-token-count features with one column per vocabulary item. Repeated context tokens increase the count. Return shape (number of examples, vocabulary_size).
Suggested tools	SciPy sparse matrices, scikit-learn vectorization utilities, NumPy, pandas, or standard Python collections. You do not need to implement a sparse-matrix class.

train_model(X_train, y_train)

Required behavior and suggested library tools are listed below. The entire function specification is kept together on this page.

Parameters	Sparse training features and target IDs
Returns	sklearn.linear_model.LogisticRegression
Requirements	Train a multiclass next-token classifier with max_iter=1000 and random_state=42.
Suggested tools	Use scikit-learn LogisticRegression. You are not expected to implement gradient descent or logistic regression from scratch.

evaluate_model(model, X, y)

Required behavior and suggested library tools are listed below. The entire function specification is kept together on this page.

Parameters	Trained model, feature matrix, and targets
Returns	dict with exactly accuracy, top_3_accuracy, and log_loss
Requirements	Use predict and predict_proba. Return Python floats. Compute top-3 accuracy using model.classes_. If fewer than three classes are represented, use the available number of classes.
Suggested tools	scikit-learn metrics such as accuracy_score, top_k_accuracy_score, and log_loss.

predict_next_token(model, context_tokens, token_to_id, id_to_token)

Required behavior and suggested library tools are listed below. The entire function specification is kept together on this page.

Parameters	Model, context-token strings, and vocabulary mappings
Returns	str
Requirements	Use only the last 3 context tokens, left-pad when necessary, map unknown tokens to <unk>, vectorize the context, and return the highest-probability token.
Suggested tools	model.predict_proba, NumPy, and vectorize_contexts.

generate_text(model, prompt, token_to_id, id_to_token, max_new_tokens=20, temperature=1.0, seed=42)

Required behavior and suggested library tools are listed below. The entire function specification is kept together on this page.

Parameters	Model, prompt, vocabulary mappings, and decoding controls
Returns	str
Requirements	Generate one token at a time. Use greedy decoding when temperature <= 0; otherwise sample from temperature-scaled probabilities with a seeded generator. Stop at <eos> or max_new_tokens. Do not include <pad>, <bos>, or <eos> in the returned text.
Suggested tools	NumPy probability and random-number utilities. You must still connect the trained model to an autoregressive generation loop.

6. Required Results and Reflection

- Print vocabulary size and the number of train, validation, and test examples.
- Print validation and test accuracy, top-3 accuracy, and log loss.
- Show at least five next-token predictions with contexts and expected targets.
- Generate at least three samples from different prompts, including one greedy sample and two temperature-based samples.
- At the end of mini_lm_project1.py, add a comment block labeled PROJECT 1 REFLECTION.
- In the reflection, identify one strength, two limitations, why bag-of-token-count features lose word order, and which pipeline components can be reused conceptually in Projects 2 and 3.

7. Testing Expectations

- Test empty strings, punctuation, capitalization, and repeated whitespace.
- Test unknown tokens and short contexts.
- Confirm examples never cross text boundaries.
- Confirm repeated context tokens create counts greater than 1.
- Confirm vocabulary IDs are deterministic.
- Confirm generation is reproducible for the same seed.
- Confirm generation stops at <eos> or max_new_tokens.
- Confirm importing mini_lm_project1.py does not train, generate, load files, or print output.

8. Five-Day Work Plan

Day	Focus	Expected progress
1	Corpus and tokens	Load the CSV, tokenize text, and inspect fixed splits.
2	Vocabulary and examples	Build vocabulary, encode text, and create context/target arrays.
3	Baseline model	Vectorize contexts, train logistic regression, and evaluate metrics.
4	Generation	Implement next-token prediction and decoding.
5	Testing and reflection	Test edge cases, write reflection comments, and review submission files.

9. Grading Rubric

The project is graded out of 100 points. Automated tests may use a corpus different from the provided sample.

Category	Points	Evidence
Corpus, tokenization, and vocabulary	20	Correct loading, deterministic tokens, special tokens, and training-only vocabulary
Examples and sparse features	20	Correct padding, boundaries, targets, shapes, and count vectors
Logistic-regression model and metrics	20	Correct library use, training, probability-based metrics, and edge-case handling
Prediction and text generation	20	Correct context handling, decoding, seed, and stopping behavior
Three-project continuity	10	Clear reflection on conceptually reusable pipeline components
Testing, reflection, and code quality	10	Evidence-based reflection, required interfaces, useful documentation, and no import-time side effects

10. Submission Checklist

Submit exactly these two files:

- mini_lm_project1.py, including the PROJECT 1 REFLECTION comment block
- mini_language_corpus.csv

No model-checkpoint, configuration, vocabulary, requirements, or notebook files are required. The Python file must import without automatically loading data, training a model, generating text, or printing output.